LeetRev - Smart LeetCode Revision System

Project Overview

LeetRev is a full-stack application designed to solve a common problem faced by DSA learners:

> "I solve problems today but forget them after a few weeks."

Instead of merely tracking solved problems, LeetRev automatically creates a personalized revision schedule and presents due problems for review using a spaced-repetition inspired workflow.

The system integrates GitHub, LeetCode, PostgreSQL, Prisma, Express, and React to create an automated learning and revision platform.

Core Idea

Traditional LeetCode trackers answer:

- How many problems have been solved?
- What is the current streak?

LeetRev answers:

- Which problems am I forgetting?
- What should I revise today?
- Which problems have been mastered?

The project automatically:

1. Reads recent GitHub commits.
2. Detects newly solved LeetCode problems.
3. Stores metadata and solutions.
4. Creates revision records.
5. Shows due reviews.
6. Reschedules reviews based on user ratings.

Tech Stack

Backend

- Node.js
- Express.js
- TypeScript
- Prisma ORM

- PostgreSQL
- Docker

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS

Data Sources

- GitHub API
- LeetCode README metadata
- LeetCode solution repository

System Architecture

GitHub Repository ↓ Ingestion Service ↓ PostgreSQL Database ↓ Express API ↓ React Frontend ↓ Revision Workflow

---

Database Design

Problem Table

Stores problem metadata.

Fields:

- id
- slug
- title
- difficulty
- topics
- url
- createdAt

Purpose:

- Central source of problem information.
- Avoids duplicate problem creation.

Solution Table

Stores solution submissions.

Fields:

- id
- problemId
- code
- language
- commitSha
- solvedAt
- timeMs
- spaceMb

Purpose:

- Tracks every solved submission.
- Stores performance metrics from commit messages.

Review Table

Stores revision schedule.

Fields:

- id
- problemId
- nextReviewAt
- intervalDays
- easeFactor
- reps
- status
- createdAt
- updatedAt

Current usage:

- nextReviewAt
- status

Future usage:

- intervalDays
- easeFactor
- reps

Status Values:

- NEW
- MASTERED

Phase 1 - GitHub Ingestion Engine ✅

Objective

Automatically import solved LeetCode problems.

Implemented

Fetch recent commits using GitHub API.

```
fetchRecentCommits()
```

Fetch files associated with each commit.

```
fetchCommitFiles()
```

Extract problem information from filenames.

Example:

1833-maximum-ice-cream-bars.cpp

Parsed into:

- slug
- language

Extract performance metrics from commit messages.

Example:

Runtime: 10ms Memory: 42MB

Parsed into:

- timeMs
- spaceMb

Fetch README metadata.

Extract:

- title
- difficulty
- URL

Store data using Prisma upserts.

Benefits:

- No duplicate entries.
- Idempotent ingestion.

Key Interview Talking Point

"We used Prisma upsert operations to make ingestion idempotent. Running the ingestion process multiple times never creates duplicate problems or solutions."

Phase 2 - Database Layer ✅

Objective

Create normalized schema.

Implemented:

Problem ↓ Solution ↓ Review

Relationships:

Problem 1 ---- N Solution

Problem 1 ---- 1 Review

Key Interview Talking Point

"The schema separates problem metadata from solution submissions and revision scheduling, making the system extensible for future analytics."

Phase 3 - Revision Queue Generation ✅

Objective

Automatically create review records.

When a new problem is detected:

```
prisma.review.upsert()
```

creates:

- problemId
- nextReviewAt

Initial scheduling:

Tomorrow at 00:00.

Key Interview Talking Point

"Every newly solved problem automatically enters the revision system without requiring manual intervention."

Phase 4 - Due Review Engine ✅

Objective

Find problems requiring revision.

Query:

- nextReviewAt <= current date
- status != MASTERED

Sorted by:

- oldest first

Limited to:

- 5 reviews

Includes:

- linked Problem information

Key Interview Talking Point

"We intentionally limit review queue size to reduce cognitive overload and maintain a focused revision workflow."

Phase 5 - REST API Layer ✅

Implemented APIs:

Reports

GET

/api/reports/yesterday

Purpose:

Returns yesterday's coding activity.

Due Reviews

GET

/api/reviews/due

Purpose:

Returns pending revision queue.

Rate Review

POST

/api/reviews/:id/rate

Body:

{ "rating":4 }

Purpose:

Updates revision schedule.

Key Interview Talking Point

"Business logic is isolated in service functions while routes remain thin controllers."

Phase 6 - Review Scheduling System ✅

Rating Model

1 → Review after 2 days

2 → Review after 3 days

3 → Review after 7 days

4 → Review after 15 days

5 → MASTERED

MASTERED problems disappear from future review queues.

Why this approach?

Instead of implementing a complex SM-2 algorithm initially, a deterministic interval-based system was chosen for simplicity and user transparency.

Key Interview Talking Point

"We prioritized a simple and explainable scheduling model before introducing more sophisticated spaced repetition algorithms."

Phase 7 - Frontend Dashboard ✅

Implemented:

Reviews Page

Displays:

- Due count
- Problem title
- Difficulty
- Due date
- Open problem link

Component Structure

Reviews.tsx

Parent component

Responsibilities:

- Fetch data
- Manage state
- Refresh queue

ReviewCard.tsx

Child component

Responsibilities:

- Render problem
- Trigger rating

React Concepts Used

- useEffect
- useState
- Props
- Callback Props
- Component Composition

Phase 8 - Real-Time Refresh Workflow ✅

Problem:

After rating, card remained visible.

Solution:

Parent-child callback pattern.

Flow:

ReviewCard ↓ onRated() ↓ Reviews.tsx ↓ loadReviews() ↓ API Refresh ↓ UI Updates

---

Key Interview Talking Point

"We used callback props to allow child components to notify parent components about successful state-changing actions."

Current Progress

Completed

✅ GitHub ingestion

✅ Problem extraction

✅ README parsing

✅ Solution storage

✅ Review creation

✅ Due review queue

✅ Rating system

✅ Revision scheduling

✅ React frontend

✅ Auto-refresh after rating

Partially Complete

🟡 Dashboard analytics

🟡 Revision history

🟡 Mastered problems page

🟡 Topic tracking

Not Started

🔲 Authentication

🔲 User accounts

🔲 Multi-user support

🔲 Email reminders

🔲 Calendar integration

🔲 Advanced spaced repetition algorithm

🔲 Performance analytics

🔲 AI-generated revision notes

Current Project Status

Estimated Completion:

Phase 1–8 Completed

≈ 65-70% of MVP complete

The application already delivers its primary value proposition:

Automatically ingest solved LeetCode problems and manage a personalized revision workflow.